

LeetCode 1248: Count Number of Nice Subarrays

BOOKING.COM

DELHIVERY

Problem Statement

Given an integer array **nums** and an integer **k**, return the number of *nice subarrays*.

A nice subarray is one that contains exactly **k** odd numbers.

Core Mathematical Logic (Sliding Window)

To find the exact number of subarrays with *exactly* **k** odd numbers, we can use a highly optimized approach leveraging the sliding window technique to find "at most" **k** odds, and subtracting the overlapping subset:

$$\text{Exactly}(K) = \text{AtMost}(K) - \text{AtMost}(K-1)$$

Solution 1: Brute Force Approach ($O(N^2)$)

This approach systematically generates all possible subarrays and counts the odd numbers within them. While structurally simple, it results in a Time Limit Exceeded (TLE) error for large arrays.

```
class Solution {
public:
    int numberOfSubarrays(vector<int>& nums, int k) {
        int n = nums.size();
        int ans = 0;

        for (int i = 0; i < n; i++) {
            int oddCount = 0;

            for (int j = i; j < n; j++) {
                if (nums[j] % 2)
                    oddCount++;

                if (oddCount == k)
                    ans++;
            }
        }
        return ans;
    }
};
```

Solution 2: Optimal Sliding Window (O(N))

By defining a helper function `atmost(nums, k)`, we can efficiently calculate the count in linear time. We use two pointers (`l` and `r`) to maintain a valid window of elements.

```
class Solution {
public:
    int n;

    int atmost(vector<int>& nums, int k){
        if (k < 0) return 0;

        int ans = 0;
        int l = 0;
        int r = 0;
        int count = 0;

        while (r < n) {
            if (nums[r] % 2 == 1) count++;

            while (count > k) {
                if (nums[l] % 2 == 1) count--;
                l++;
            }

            ans += (r - l + 1);
            r++;
        }
        return ans;
    }

    int numberOfSubarrays(vector<int>& nums, int k) {
        n = nums.size();
        return atmost(nums, k) - atmost(nums, k - 1);
    }
};
```